

# Q1 – 10 marks

## Basic testcases – 6 marks

- **1 mark** each – total **3 marks**
- Just checking the format for procinfo, fsinfo here as specified below
- meminfo should print memory left around 128MB (RAM as specified in the Makefile, accepting solutions close to this)
- Allowing both "memory left", "physical memory left" in meminfo

Shell

```
$ cat procinfo
1 sleep  init
2 sleep  sh
3 run    cat
```

Shell

```
$ cat meminfo
{memory} left: {mem:d}B
```

Shell

```
$ cat fsinfo
pid: 1
open FDs: 0 1 2
pid: 2
open FDs: 0 1 2
pid: 3
open FDs: 0 1 2 3
#files with refcnt > 1: {num_refcnt}
Bcache hits: {bch1}, Bcache misses: {bcm1}
```

- Shouldn't be able to write into procfs files, **0.5 marks** each – total **1.5 marks**

Shell

```
$ echo hi > procinfo
open procinfo failed
```

```
$ echo hi > meminfo
open meminfo failed
$ echo hi > fsinfo
open fsinfo failed
```

- p1d is a combination of the above two testcases, but uses syscalls directly, this was to ensure that cat, echo were not modified to get the solution. If p1d fails to print the output in the correct format shown as above for procinfo, meminfo, fsinfo, 0 marks are given for testcases using the respective procfs file. It's okay if all procfs files don't work, marks are awarded for each working procfs file separately.
- p1d has **0.5 marks** for each working file – total **1.5 marks**

## meminfo – 1 mark

Should decrease by around 4MB

```
Shell
$ p1a &
$ cat meminfo
{memory} left: {mem1:d}B
$ p1a &
$ cat meminfo
{memory} left: {mem2:d}B
```

## fsinfo – 3 marks

### Open FDs for each process – 1 mark

- p1b, p1c are 0.5 marks each -- total 1 mark
- Only checking if open FDs are correctly printed here
- Outputs are as given in the question paper.

### Bcache hits, misses – 1 mark

- 0.5 if #bcache hits increase, i.e bch1 <= bch2 <= bch3
- 0.5 if #bcache misses increase when opening a new file, but remain the same when that same file is opened again. i.e bcm1 <= bcm2 = bcm3

- Only bcache misses, hits are checked in this testcase

```
Shell
$ cat fsinfo
pid: 1
open FDs: 0 1 2
pid: 2
open FDs: 0 1 2
pid: 3
open FDs: 0 1 2 3
#files with refcnt > 1: {refcnt1}
Bcache hits: {bch1:d}, Bcache misses: {bcm1:d}
$ cat README
{README1}
$ cat fsinfo
pid: 1
open FDs: 0 1 2
pid: 2
open FDs: 0 1 2
pid: 5
open FDs: 0 1 2 3
#files with refcnt > 1: {refcnt2}
Bcache hits: {bch2:d}, Bcache misses: {bcm2:d}
$ cat README
{README2}
$ cat fsinfo
pid: 1
open FDs: 0 1 2
pid: 2
open FDs: 0 1 2
pid: 7
open FDs: 0 1 2 3
#files with refcnt > 1: {refcnt3}
Bcache hits: {bch3:d}, Bcache misses: {bcm3:d}
```

New testcase! p1e: #files with refcnt > 1 – 1 mark

- Here's user/p1e.c used for this testcase:

```
C/C++
#include "kernel/types.h"
#include "user.h"
#include "kernel/fcntl.h"
```

```

int main() {
    if (open("README", O_RDONLY) == -1) {
        printf("couldn't open README");
        exit(1);
    }
    fork();
    while (1);
    exit(0);
}

```

- expected output:

```

Shell
$ cat fsinfo
pid: 1
open FDs: 0 1 2
pid: 2
open FDs: 0 1 2
pid: 3
open FDs: 0 1 2 3
#files with refcnt > 1: 1
Bcache hits: {bch1}, Bcache misses: {bcm1}
$ p1e &
$ cat fsinfo
pid: 1
open FDs: 0 1 2
pid: 2
open FDs: 0 1 2
pid: 6
open FDs: 0 1 2 3
pid: 5
open FDs: 0 1 2 3
pid: 7
open FDs: 0 1 2 3
#files with refcnt > 1: 2
Bcache hits: {bch2}, Bcache misses: {bcm2}

```

## Q2 – 10 marks

Symlink functionality:

- Correct file types – **2 marks**
- Correct inode reference counts – **2 marks**
- Write redirection working – **2 marks**
- Read redirection working – **2 marks**
- Removing symlink does not remove target file – **2 marks**

*\* Second functionality is only checked if the first functionality works, the rest are only checked if second functionality works*

C/C++

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "kernel/param.h"
#include "kernel/fcntl.h"
#include "user/user.h"

#define TARGET_FILE "sl_target.txt"
#define LINK_FILE "sl_link.txt"

static void
cleanup(void)
{
    unlink(LINK_FILE);
    unlink(TARGET_FILE);
}

static int
write_all(int fd, const char* s, int n)
{
    return write(fd, s, n) == n;
}

static int
make_target_with_content(const char* s, int n)
{
    int fd = open(TARGET_FILE, O_CREATE | O_WRONLY | O_TRUNC);
    if(fd < 0)
        return 0;

    if(!write_all(fd, s, n)) {
        close(fd);
        return 0;
    }
}
```

```

    }

    close(fd);
    return 1;
}

int
main(void)
{
    int score = 0;
    int fd;
    int n;
    char buf[64];
    // char linkbuf[MAXPATH];
    const char* s1 = "alpha";
    const char* s2 = "beta";

    cleanup();

    // Test 1 (2 pts): create symlink and verify symlink type, and confirm readlink.
    if(make_target_with_content(s1, 5) && symlink(TARGET_FILE, LINK_FILE) == 0 &&
        filetype(LINK_FILE) == T_SYMLINK)
        score += 2;

    // Test 2 (2 pts): refcount should stay 1 for both target and symlink inode.
    if(score == 2) {
        if(inoderefcnt(TARGET_FILE) == 1 && inoderefcnt(LINK_FILE) == 1)
            score += 2;
    }

    // Test 3 (2 pts): reading via symlink should read target content.
    if(score >= 4) {
        fd = open(LINK_FILE, O_RDONLY);
        if(fd >= 0) {
            memset(buf, 0, sizeof(buf));
            n = read(fd, buf, sizeof(buf));
            close(fd);
            if(n == 5 && memcmp(buf, s1, 5) == 0)
                score += 2;
        }
    }

    // Test 4 (2 pts): writing via symlink should update target file.
    if(score >= 4) {
        fd = open(LINK_FILE, O_WRONLY | O_TRUNC);
    }
}

```

```

if(fd >= 0) {
    if(write_all(fd, s2, 4)) {
        close(fd);
        fd = open(TARGET_FILE, O_RDONLY);
        if(fd >= 0) {
            memset(buf, 0, sizeof(buf));
            n = read(fd, buf, sizeof(buf));
            close(fd);
            if(n == 4 && memcmp(buf, s2, 4) == 0)
                score += 2;
        }
    } else
        close(fd);
}
}

// Test 5 (2 pts): removing symlink should not remove target file.
if(score >= 4) {
    if(unlink(LINK_FILE) == 0) {
        fd = open(TARGET_FILE, O_RDONLY);
        if(fd >= 0) {
            close(fd);
            score += 2;
        }
    }
}

cleanup();

printf("GRADE_SCORE:%d\n", score);
exit(0);
}

```

## Q3 – 10 marks

### a. itests txoff is working - 3 marks

```
Shell
$ itests txoff
Testing busy-waiting behavior of UART interrupts.
[TX off, RX on]
Press Enter to start this setup...

[read]
Type "1234" and press Enter. Waiting up to 100 ticks...
1234
SUCCESS: read completed within timeout.

[write]
Writing "1234". Waiting up to 100 ticks...
1234
SUCCESS: write completed within timeout.
```

### b. itests rxoff is working - 5 marks

```
Shell
$ itests rxoff
Testing busy-waiting behavior of UART interrupts.

[RX off, TX on]
Press Enter to start this setup...

[read]
Type "1234" and press Enter. Waiting up to 100 ticks...
1234

SUCCESS: read completed within timeout.

[write]
Writing "1234". Waiting up to 100 ticks...
1234

SUCCESS: write completed within timeout.
```

- if ( a and b ) is correctly done; then -
- c. itests alloff and toggle alloff is working - **1 mark**

```
C/C++
$ itests alloff
Testing busy-waiting behavior of UART interrupts.

[All off]
Press Enter to start this setup...

[read]
Type "1234" and press Enter. Waiting up to 100 ticks...
1234

SUCCESS: read completed within timeout.

[write]
Writing "1234". Waiting up to 100 ticks...
1234

SUCCESS: write completed within timeout.
$ toggle alloff
$ echo Hello
Hello
$
```

- d. itests allon and toggle allon is working - **1 mark**

```
C/C++
$ itests allon
Testing busy-waiting behavior of UART interrupts.

[All on]
Press Enter to start this setup...

[read]
Type "1234" and press Enter. Waiting up to 100 ticks...
1234

SUCCESS: read completed within timeout.
```

```
[write]  
Writing "1234". Waiting up to 100 ticks...  
1234
```

```
SUCCESS: write completed within timeout.  
$ toggle allon  
$ echo Hello  
Hello  
$
```